

Introduction to Python for Business Statistics

A 6-Week Intensive Course

Expanded Edition

This expanded edition accompanies a 6-week intensive course designed for graduate certificate students in Business Intelligence and Data Analytics. Python concepts are introduced through business statistics — students build both skills simultaneously, each reinforcing the other. This edition adds extended conceptual background, worked examples with output, and short business mini case studies to every chapter.

© 2026 Patrick Dolinger — Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0)

Week	Python	Statistics
1	Variables, types, arithmetic	Data types, measurement scales
2	Lists, NumPy arrays	Central tendency, samples vs populations
3	Conditionals, loops, comprehensions	Frequency distributions, outlier detection
4	Functions, error handling, Pandas	Variance, std deviation, z-scores, IQR
5	Data cleaning, Matplotlib, Seaborn	Distributions, correlation, visualisation
6	groupby, pivot tables, EDA workflow	Grouped analysis, coefficient of variation

How To Use This Book

The course pairs Python and statistics on purpose. Most introductory Python texts teach the language in isolation — variables, types, loops — and leave it to the student to find a context to apply it. Most introductory statistics texts assume the calculations will be done in a calculator, in Excel, or by hand, and treat the computational tooling as an afterthought. Neither approach reflects how data work is actually done in industry.

Every section in this book follows the same rhythm:

1. **A statistical idea is introduced** — what it measures, when it applies, and the reasoning behind it.
2. **The Python tool is introduced** — the syntax and library calls that compute or visualise it.
3. **A worked example is shown** — small enough to follow line by line, with the output included.
4. **A business interpretation is offered** — what the number actually tells a stakeholder.

Read the worked examples with a Python session open. Type them in. Change the inputs. The single most reliable predictor of whether a student finishes this course confident is whether they ran the code, not whether they read it.

The chapters are cumulative. Chapter 4's discussion of variance assumes you understand the mean from Chapter 1 and arrays from Chapter 2. Chapter 6's grouped analysis assumes you have fluency with Pandas DataFrames from Chapter 4 and visualisation from Chapter 5. Skipping ahead is rarely productive.

Table of Contents

- Chapter 1 — Data, Variables, and Types
 - Chapter 2 — Lists, Arrays, and Central Tendency
 - Chapter 3 — Conditionals, Loops, and Frequency Distributions
 - Chapter 4 — Functions, Error Handling, and an Introduction to Pandas
 - Chapter 5 — Data Cleaning and Visualisation
 - Chapter 6 — Grouped Analysis, Pivot Tables, and the EDA Workflow
 - License
-

Chapter 1 — Data, Variables, and Types

The first chapter is foundational in two senses. It introduces the syntax you will use in every later chapter — assigning variables, doing arithmetic, formatting output — and it introduces a way of thinking about data that will shape every analysis you do. Before you write code that calculates something, you have to know whether the calculation is meaningful for the kind of data you are calculating it on.

1.1 What Is Data?

Data is recorded information. In a business context, data is collected whenever something is measured, counted, or categorised — a sale is recorded, a customer leaves a rating, a shipment is logged, a page is loaded, a button is clicked.

That definition sounds trivial, but it has a useful implication. Data is a *record* of something that already happened. It is not the thing itself. A row in a sales database is not a sale; it is a description of a sale, frozen at the moment it was written down. Whether that description is accurate, complete, or useful depends entirely on how the record was made — what was captured, what was left out, and what was assumed.

This is why data analysts spend so much time on what looks, from the outside, like clerical work: cleaning, validating, reformatting. The goal is not just to make the data look tidy. It is to make sure the recorded version of reality is good enough to support the decisions you intend to make from it.

Before writing a single line of Python, it helps to think about what kind of data you are working with. Not all data is the same, and the type of data determines what you can do with it.

A Working Example

Imagine a small online retailer. Every time a customer completes a checkout, a row is added to a sales database. A single row might look like this:

Field	Value
transaction_id	10042
customer_id	8133
product_name	"Laptop Stand"
sales_region	"Northeast"
units_sold	2
unit_price	49.95
discount_pct	0.10
revenue	89.91
is_returned	False
transaction_date	"2026-03-14"

Every column is a piece of data, but they are not all the same kind of data. `transaction_id` is a number, but it is being used as a label — averaging two transaction IDs together gives you a number, but the number is meaningless. `revenue` is also a number, but averaging revenue across rows gives you the very useful concept of an average sale value. The Python `float` type cannot tell these two situations apart. You can.

1.2 Measurement Scales

Every variable in a dataset belongs to one of four **measurement scales**. This concept comes from statistics — it was articulated by the psychologist Stanley Smith Stevens in the 1940s — but it has direct practical consequences in Python. The scale tells you which operations and analyses are valid for that variable.

Scale	Description	Example
Nominal	Categories with no meaningful order	Product name, sales region, payment method
Ordinal	Categories with a meaningful order, but unequal gaps	Customer satisfaction (1–5 stars), performance rating
Interval	Ordered with equal gaps, but no true zero	Year, temperature in °C
Ratio	Ordered with equal gaps and a true zero	Revenue, units sold, profit margin

Each scale supports the operations of the scales below it but adds new ones. Nominal data supports counting only — you can ask *how many* transactions happened in each region. Ordinal data supports ordering on top of counting — you can ask *which* satisfaction rating was most common, and you can sort customers from least to most satisfied. Interval data supports addition and subtraction — the difference between 20°C and 30°C is the same as between 30°C and 40°C. Ratio data supports all four arithmetic operations, including meaningful ratios — 200 of revenue is twice as much as 100 of revenue. You cannot say that 40°C is twice as hot as 20°C, because the zero on the Celsius scale is arbitrary.

The key distinction between interval and ratio is the **true zero**. A true zero means the value zero represents a genuine absence of the thing being measured. Revenue of \$0 means no revenue was earned — that is a true zero. The year 0 does not mean "no time" — so year is interval, not ratio. Temperature in degrees Celsius has a zero set by the freezing point of water, which is a useful reference but not the absence of heat. Temperature in Kelvin has a true zero (absolute zero, the absence of thermal energy), so Kelvin is ratio scale even though Celsius is not.

Why it matters: You can calculate the mean of a ratio variable (average revenue), but calculating the mean of a nominal variable (average product name) is meaningless. Calculating the mean of an ordinal variable (average star rating) is technically possible but statistically questionable, since the gaps between 1 and 2 stars may not equal the gap between 4 and 5 stars. The Python interpreter will happily compute averages of any numeric column. It is your job to know whether the answer means anything.

Mini Case Study — A Spreadsheet With Hidden Scale Errors

A junior analyst is asked to summarise a customer survey. The dataset has three columns: `customer_id` (an integer like 8133), `region_code` (1, 2, 3, or 4 mapping to four sales regions), and `satisfaction` (an integer 1 through 5). The analyst computes the mean of every numeric column and presents the results in a one-page summary:

Average customer ID: 8429. Average region code: 2.4. Average satisfaction: 3.8.

The first two of those three numbers are nonsense. The customer ID is nominal — the integers are labels, not quantities. The region code is also nominal — it has been *encoded* as a number for storage convenience, but the encoding does not give the values any arithmetic meaning. Only the third number, the average satisfaction, is even potentially meaningful, and even there a careful analyst would mention that satisfaction is ordinal and the assumption of equal gaps between ratings is conventional rather than rigorous.

This is a real and recurring failure mode in early data work. Python and Excel will both compute averages of anything numeric. Recognising which computations are legitimate is a statistical skill, not a Python skill.

1.3 Python Variables

A **variable** is a named container that stores a value. You create one using the assignment operator `=`.

```
sale_amount = 349.95
```

The variable name goes on the left, the value on the right. From this point forward, anywhere you use `sale_amount` in your code, Python substitutes the value `349.95`.

It helps to read the assignment operator left-to-right, as in "*saleamount becomes 349.95*" or "*assign 349.95 to saleamount*." Reading it as "equals" causes confusion later, when you encounter the equality test `==`. The `=` symbol in Python does not assert that two things are equal; it commands the interpreter to make a name refer to a value.

A variable is not a box. The name `sale_amount` is a label that points at a value stored somewhere in memory. When you reassign `sale_amount = 400.00`, you are not modifying the original 349.95 — you are pointing the label at a new value. This becomes important later when working with lists and DataFrames, where multiple names can point at the same underlying object.

Naming Rules

Variable names in Python must follow these rules:

- Start with a letter or underscore, never a number
- Contain only letters, numbers, and underscores
- Cannot be a Python reserved word (such as `if`, `for`, `True`)

By convention, Python variable names use **snake_case** — all lowercase with underscores between words. This is the standard in data work and is enforced by every major Python style guide.

```
# Good names
sale_amount = 349.95
product_name = "Laptop Stand"
is_returned = False

# Avoid
SaleAmount = 349.95      # not snake_case (this is a class-naming convention)
sa = 349.95              # too short, not descriptive
sale-amount = 349.95     # syntax error – hyphens are not allowed
1st_sale = 349.95        # syntax error – cannot start with a digit
```

A useful guideline: a variable name should be readable by someone who has not seen your code before. `sa` is faster to type than `sale_amount`, but it costs the reader two extra seconds of cognitive load every time it appears. When you have a hundred variables in a script, that adds up.

Worked Example

```
product_name = "Laptop Stand"
units_sold   = 2
unit_price   = 49.95
discount_pct = 0.10

revenue = units_sold * unit_price * (1 - discount_pct)
print(revenue)
```

Output:

```
89.91
```

Note that nothing is displayed until `print()` is called. Assigning a value to a variable does not produce visible output — it stores the value silently. This is one of the most common stumbling blocks for beginners moving from Excel: in a spreadsheet, every formula updates a visible cell, while in Python, calculations happen invisibly until you ask for them.

1.4 Core Data Types

Python has four data types you will use in almost every analysis. The interpreter assigns a type automatically based on how a value is written.

Integer — `int`

Whole numbers. Used for counts, IDs, and quantities that cannot be fractional.

```
units_sold = 150
transaction_id = 1042
fiscal_year = 2023
```

Python integers have no fixed size limit (unlike most other languages). You will not run out of bits even if you store a number with hundreds of digits.

Float — `float`

Decimal numbers. Used for prices, rates, percentages, and most financial values.

```
sale_amount = 349.95
tax_rate = 0.13
margin = 0.4275
```

Floats are stored using a binary approximation called IEEE 754. This means some decimal values cannot be represented exactly:

```
print(0.1 + 0.2)
```

Output:

```
0.30000000000000004
```

This is not a Python bug — every language with floating-point arithmetic has the same behaviour. For business reporting, round to a sensible number of decimal places using `round()` or an f-string format specifier. For exact decimal arithmetic (e.g., currency at a financial institution), Python's `decimal` module is available, but it is rarely needed at the analyst level.

String — `str`

Text data. Used for names, categories, labels, and any value that is not a number. Always enclosed in quotes — single or double, your choice, but be consistent.

```
product_name = "Laptop Stand"
sales_region = "Northeast"
payment_method = "Credit Card"
```

A common mistake is to store numeric data as a string by accident — for example, when reading data from a file with a number wrapped in quotes. Strings cannot be averaged or summed, so the failure shows up the moment you try to do statistics on the column.

Boolean — `bool`

True or false values. Used for flags and binary conditions.

```
is_returned = False
is_online_sale = True
is_discounted = False
```

Note that `True` and `False` are capitalised in Python. Lowercase `true` and `false` are not valid keywords and produce a `NameError`. Booleans behave as integers in arithmetic — `True` is 1 and `False` is 0 — which is occasionally useful for counting how many rows in a column meet a condition.

Checking a Variable's Type

Use the built-in `type()` function to confirm what type a variable is. This is invaluable when debugging unexpected behaviour.

```
print(type(sale_amount))    # <class 'float'>
print(type(units_sold))    # <class 'int'>
print(type(product_name))  # <class 'str'>
print(type(is_returned))   # <class 'bool'>
```

Connecting Types to Measurement Scales

Python types and statistical measurement scales are related but not identical. The same Python type can represent different scales depending on context.

Scale	Typical Python Type	Notes
Nominal	<code>str</code> , sometimes <code>int</code>	An <code>int</code> used as a postal code or product ID is nominal

Scale	Typical Python Type	Notes
Ordinal	<code>int</code> or <code>str</code>	A rating stored as <code>int</code> is still ordinal — the gaps are unequal
Interval	<code>int</code> or <code>float</code>	Year is <code>int</code> , but is interval scale
Ratio	<code>int</code> or <code>float</code>	Revenue, units sold, quantities

The type tells Python how to store the value. The scale tells you what statistical operations are valid. The two are independent. A careful analyst tracks the scale of every column mentally, even though Python itself does not.

1.5 Arithmetic Operations

Python supports standard arithmetic. The results are used constantly in business analysis — calculating totals, averages, growth rates, and proportions.

Operator	Operation	Example	Result
<code>+</code>	Addition	<code>10 + 3</code>	<code>13</code>
<code>-</code>	Subtraction	<code>10 - 3</code>	<code>7</code>
<code>*</code>	Multiplication	<code>10 * 3</code>	<code>30</code>
<code>/</code>	Division	<code>10 / 3</code>	<code>3.333...</code>
<code>//</code>	Floor division	<code>10 // 3</code>	<code>3</code>
<code>%</code>	Remainder	<code>10 % 3</code>	<code>1</code>
<code>**</code>	Exponentiation	<code>10 ** 3</code>	<code>1000</code>

Note that `/` always returns a `float`, even if the result is a whole number. `//` returns an `int` by discarding the decimal. The remainder operator `%` is genuinely useful in business contexts — it tells you whether a number is divisible by another, which comes up in scheduling, batching, and reporting periods.

Order of Operations

Python follows standard mathematical precedence: exponentiation first, then multiplication and division, then addition and subtraction. Use parentheses generously — they cost nothing and make intent unmistakable.

```
# Hard to read
total = price * 1 + tax_rate * units

# Clear
total = (price * (1 + tax_rate)) * units
```

Worked Example — Annual Revenue

```
q1_revenue = 12400.00
q2_revenue = 15800.00
q3_revenue = 13950.00
q4_revenue = 19200.00

annual_revenue = q1_revenue + q2_revenue + q3_revenue + q4_revenue
print(annual_revenue)
```

Output:

```
61350.0
```

Growth Rate Calculation

A common business calculation is the percentage change between two values:

$\text{pct change} = \frac{\text{new} - \text{old}}{\text{old}} \times 100$

```
old_revenue = 48000
new_revenue = 54600

growth_pct = ((new_revenue - old_revenue) / old_revenue) * 100
print(growth_pct)
```

Output:

```
13.75
```

This formula appears so often that we will turn it into a reusable function in Chapter 4.

1.6 The Mean

The **mean** is the most commonly used measure of central tendency. It is the sum of all values divided by the count of values.

$$\bar{x} = (\sum x) / (n)$$

Where $\bar{x}$ is the mean, $\sum x$ is the sum of all values, and n is the number of values.

```
average_quarterly_revenue = annual_revenue / 4
print(average_quarterly_revenue)
```

Output:

```
15337.5
```

The mean is appropriate for **interval** and **ratio** scale variables. Applying it to nominal or ordinal data produces a number, but that number has no meaningful interpretation.

The mean has one notable weakness: every value contributes to it equally, including unusual ones. A single very large or very small value can pull the mean noticeably away from where most of the data sits. This is why Chapter 2 introduces the median as an alternative and Chapter 3 introduces formal outlier detection. For now, it is enough to remember that the mean is informative when the data is roughly symmetric and unhelpful when it is dominated by a few extreme values.

1.7 String Operations

Strings represent categorical and text data. Python provides several tools for working with them cleanly.

Common String Methods

```
product = "laptop stand"

print(product.upper())    # LAPTOP STAND
print(product.title())    # Laptop Stand
print(product.lower())    # laptop stand
print(len(product))       # 12
```

`upper()`, `title()`, and `lower()` are particularly useful when cleaning categorical data. Real-world text data is wildly inconsistent — `"Northeast"`, `"northeast"`, `"NORTHEAST"`, and `" Northeast "` may all refer to the same region but are different strings to Python. Standardising to one case before grouping is a routine first step.

Concatenation

Strings can be joined using `+`.

```
product = "Laptop Stand"
region = "Northeast"
label = product + " - " + region
print(label)    # Laptop Stand - Northeast
```

Concatenation only works between strings. Trying to concatenate a string with a number raises a `TypeError`. Convert the number first using `str()`:

```
units = 47
label = "Units sold: " + str(units)
```

This works, but the f-string approach below is much cleaner and is what you should use in practice.

f-Strings

f-strings (introduced in Python 3.6) are the cleanest way to embed variable values inside a text output. Prefix the string with `f` and place variable names inside `{}`.

```
product = "Laptop Stand"
units = 47
region = "Northeast"

print(f"{product} sold {units} units in {region}.")
```

Output:

```
Laptop Stand sold 47 units in Northeast.
```

f-strings also support formatting specifiers. The most useful for business data is `:.2f`, which rounds a float to two decimal places. `:,.2f` adds thousands separators.

```
revenue = 8420.5
print(f"Revenue: ${revenue:.2f}")           # Revenue: $8420.50
print(f"Revenue: ${revenue:,.2f}")          # Revenue: $8,420.50
print(f"Growth: {0.1375:.1%}")              # Growth: 13.8%
```

The `%` specifier multiplies by 100 and appends a percent sign in one step — useful for percentage reporting.

1.8 Mini Case Study — A One-Row Sales Summary

Putting the chapter together, suppose you are asked to produce a one-line summary of a single transaction for an end-of-day report. The transaction data looks like this:

```
product_name = "Laptop Stand"
sales_region = "Northeast"
units_sold   = 2
unit_price   = 49.95
discount_pct = 0.10

# Calculations
gross      = units_sold * unit_price
revenue    = gross * (1 - discount_pct)

# Formatted summary
summary = (f"{units_sold} units of {product_name} sold in {sales_region} "
           f"at ${unit_price:.2f} each "
           f"({discount_pct:.0%} discount) – net revenue ${revenue:,.2f}")

print(summary)
```

Output:

```
2 units of Laptop Stand sold in Northeast at $49.95 each (10% discount) – net revenue $89.91
```

This single example exercises every concept from the chapter — variables, arithmetic, a mix of types, and formatted string output. In Chapter 4 we will turn this kind of calculation into a reusable function and apply it to many transactions at once. For now, focus on getting comfortable with the syntax.

1.9 Chapter Summary

Concept	Key Point
Measurement scales	Nominal, ordinal, interval, ratio — determines valid operations
Variables	Named containers assigned with <code>=</code> , use <code>snake_case</code>
<code>int</code>	Whole numbers — counts, IDs, years
<code>float</code>	Decimal numbers — prices, rates, percentages
<code>str</code>	Text — names, categories, labels
<code>bool</code>	True/False — flags and binary conditions
<code>type()</code>	Returns the data type of any variable
Mean	Sum divided by count — valid for interval and ratio only
f-strings	Clean way to embed variables in printed output

Review Questions

1. A dataset contains a column called `customer_id` stored as integers (e.g., 10042, 10043). What measurement scale is this variable? Can you calculate its mean? Why or why not?
2. A sales rep's performance is rated as Bronze, Silver, or Gold. What measurement scale is this? What Python type would you use to store it?
3. What is the difference between `/` and `//` in Python? Give an example where the distinction matters.
4. Write a Python statement that calculates the percentage change in revenue from 48,000 to 54,600.
5. A colleague argues that you can average customer star ratings (1–5) to get a "mean satisfaction score." What is the statistical argument against this? When might you do it anyway?

Chapter 2 — Lists, Arrays, and Central Tendency

In Chapter 1, every variable held one value. That is enough to do calculations on a single transaction, but real datasets contain thousands or millions of rows. Working with them one variable at a time is impossible. This chapter introduces two structures for storing collections of values — the Python list and the NumPy array — and three statistical measures that summarise a collection with a single number.

2.1 From Single Values to Collections

In Week 1 we stored one value per variable. Real data rarely works that way. A sales dataset has hundreds of transactions. A monthly report has twelve figures. An employee file has one row per person. A web log has one row per page request, accumulating millions of rows per day.

A spreadsheet handles collections naturally — every column is a sequence of values, and every formula operates on a range. Python needs an explicit data structure to represent the same idea. There are several options, and the choice between them affects both convenience and performance.

This chapter covers the two structures you will use most in data work: the **list**, which is general-purpose, and the **NumPy array**, which is specialised for numerical computation and is the foundation of nearly everything in the Python data ecosystem.

2.2 Python Lists

A **list** is an ordered collection of values stored in a single variable. Values are enclosed in square brackets and separated by commas.

```
monthly_sales = [14200, 13850, 16400, 15900, 17200, 18100]
products = ["Laptop Stand", "Monitor Arm", "Keyboard Tray", "Cable Manager"]
```

Lists are *ordered* in two senses. First, the items appear in a specific sequence — the first item is fixed at position 0, the second at position 1, and so on. Second, that order is preserved across operations: looping through a list visits the items in the same order every time. This is unlike some other collection types (such as the `set`) where order is not guaranteed.

Lists can hold any data type — integers, floats, strings, or a mix. In data work, a single list typically holds one type consistently, mirroring a column in a spreadsheet. A list of mixed types is legal but is usually a sign that the data has not been cleaned properly.

Useful List Functions

Function	What It Does	Example
<code>len()</code>	Count of items	<code>len(monthly_sales)</code> → 6
<code>sum()</code>	Sum of numeric values	<code>sum(monthly_sales)</code> → 95650
<code>min()</code>	Smallest value	<code>min(monthly_sales)</code> → 13850
<code>max()</code>	Largest value	<code>max(monthly_sales)</code> → 18100
<code>sorted()</code>	Returns a sorted copy	<code>sorted(monthly_sales)</code>

These five functions cover a surprising amount of basic analysis. Note that `sorted()` returns a *new* list, leaving the original unchanged. This is a small but important detail — when you sort a list to compute the median (which we will do shortly), you usually want the original order preserved for later use.

Modifying Lists

Lists support several useful methods for adding and removing items:

```
monthly_sales = [14200, 13850, 16400]
monthly_sales.append(15900)           # add to the end
monthly_sales.append(17200)
monthly_sales.append(18100)
print(monthly_sales)
```

Output:

```
[14200, 13850, 16400, 15900, 17200, 18100]
```

Other useful methods: `insert(index, value)` to place an item at a specific position, `remove(value)` to delete the first occurrence of a value, and `pop()` to remove and return the last item. The append-in-a-loop pattern is so common that we will see it again in Chapter 3.

2.3 Indexing

Every item in a list has a position called an **index**. Python indices start at **0**.

```
monthly_sales = [14200, 13850, 16400, 15900, 17200, 18100]
index:          0         1         2         3         4         5
```

Access an item by placing its index in square brackets.

```
monthly_sales[0]    # 14200 – January
monthly_sales[2]    # 16400 – March
monthly_sales[5]    # 18100 – June
```

Zero-based indexing trips up many beginners. The first item is at index 0, not 1. The last item of a six-element list is at index 5, not 6. Asking for `monthly_sales[6]` raises an `IndexError`. This is a quirk of nearly every modern programming language and is mostly a historical artefact, but it is the convention you have to live with.

Negative Indexing

Negative indices count from the end of the list. This is useful when you want the last item without knowing the list's length.

```
monthly_sales[-1]   # 18100 – last item (June)
monthly_sales[-2]   # 17200 – second to last (May)
```

A common idiom is `data[-1]` to grab "the most recent" value from a chronologically-ordered list — useful in time series work.

2.4 Slicing

A **slice** extracts a portion of a list. The syntax is `list[start:stop]`, where the stop index is **not included**.

```
monthly_sales[0:3]    # [14200, 13850, 16400] – January to March
monthly_sales[3:6]    # [15900, 17200, 18100] – April to June
```

The stop-exclusive convention is another inheritance from C-family languages and seems arbitrary at first. It does have a practical benefit: `data[a:b]` returns exactly `b - a` items, which makes length calculations easy. It also means consecutive slices like `data[0:3]` and `data[3:6]` partition the list cleanly, with no overlap and no gap.

Omitting either boundary defaults to the start or end of the list.

```
monthly_sales[:3]     # first three items – same as [0:3]
monthly_sales[3:]     # from index 3 to the end
monthly_sales[:]      # a full copy of the list
```

Slicing also accepts a step: `data[::-2]` returns every other item; `data[::-1]` returns the list reversed. These are powerful but used sparingly in business data work.

This maps naturally to business data. Extracting a quarter, a fiscal half-year, or the most recent period from a list is a slicing operation. Once we move to Pandas in Chapter 4, we will see the same idea applied to rows and columns of a table.

2.5 NumPy Arrays

The Python list is general-purpose. For numerical data, **NumPy** (Numerical Python) provides a more powerful structure: the **array**.

NumPy arrays are faster than lists for numerical work and support operations that would otherwise require loops. They are the foundation of most data analysis in Python — Pandas DataFrames, scikit-learn models, and most scientific libraries are built on top of NumPy arrays. Investing time in understanding them pays dividends.

Import NumPy using the standard alias `np`. This convention is almost universal; you will see it in every textbook, every Stack Overflow answer, and every production codebase.

```
import numpy as np

sales_array = np.array([14200, 13850, 16400, 15900, 17200, 18100])
print(sales_array)
```

Output:

```
[14200 13850 16400 15900 17200 18100]
```

Notice the printed representation has no commas — this is one quick way to tell at a glance whether you are looking at a list or an array.

Element-Wise Arithmetic

The most immediate advantage of NumPy arrays is **element-wise arithmetic** — an operation applied to the whole array at once, without needing to loop through each item. This is sometimes called *vectorisation*.

```
# Apply a 10% revenue increase to every month
adjusted = sales_array * 1.10
print(adjusted)
```

Output:

```
[15620. 15235. 18040. 17490. 18920. 19910.]
```

```
# Subtract a fixed monthly overhead
net = sales_array - 2000
print(net)
```

Output:

```
[12200 11850 14400 13900 15200 16100]
```

With a plain Python list, each of these would require a loop. With a NumPy array, one line handles it. This is a recurring pattern in data work — most operations on a column do not actually need a loop in your code, even if conceptually they "do something to every row." NumPy and Pandas push the loop into highly optimised compiled code, which is both faster and harder to get wrong.

Lists vs Arrays

	Python List	NumPy Array
Holds mixed types	Yes	No — all elements must be the same type
Element-wise math	No — requires a loop	Yes — direct operations
Speed on large data	Slow	Fast
Statistical functions	Limited	Extensive (<code>np.mean</code> , <code>np.std</code> , etc.)
Memory efficiency	Higher overhead per item	Compact, fixed-size storage

Use lists for general-purpose storage — collections of strings, mixed types, or small data that does not need numerical operations. Use NumPy arrays whenever you are doing numerical computation on more than a handful of values.

The performance gap is large. On an array of a million numbers, a NumPy operation typically runs 50 to 100 times faster than the equivalent Python loop. On six numbers, the difference is invisible. The benefit accrues with scale.

2.6 Samples and Populations

Before computing any statistics, it is worth being clear about what your data represents.

A **population** is the complete set of all cases you are interested in. A **sample** is a subset of that population — the data you actually have.

	Population	Sample
Definition	All cases of interest	A subset of the population
Size notation	N	n
Mean notation	μ (mu)	$\bar{x}$ (x-bar)

In most real business scenarios, you are working with a **sample**. Six months of sales data is a sample from the company's full sales history. A survey of 500 customers is a sample from all customers. Even a "complete" dataset is usually a sample of a broader, time-extended population — last quarter's transactions are a sample drawn from the company's eventual long-run transaction history.

Populations are often unobservable in practice. We rarely measure every customer, every transaction across all time, or every shipment a logistics company will ever handle. Statistics is, in large part, the discipline of using a sample to make defensible claims about the population it was drawn from. The size of the sample, how it was selected, and how representative it is of the population all affect how confidently those claims can be made.

This distinction matters because some statistical formulas differ slightly between sample and population calculations. You will see this directly in Chapter 4 when we cover variance and standard deviation, where the *sample* version of the formula divides by $n-1$ instead of n to correct for a subtle bias.

2.7 Measures of Central Tendency

A **measure of central tendency** summarises a dataset with a single value representing its centre. There are three standard measures: the mean, the median, and the mode. Each has a use case where it is the right choice and another where it would mislead.

The Mean

The mean is the sum of all values divided by the count.

$$\bar{x} = (\sum x)/(n)$$

```
import numpy as np
sales_array = np.array([14200, 13850, 16400, 15900, 17200, 18100])

mean = np.mean(sales_array)
print(mean)
```

Output:

```
15941.666666666666
```

The mean uses every value in the dataset. This makes it precise but sensitive — a single extreme value can shift it significantly. It is the right summary when the data is roughly symmetric and free of outliers.

The Median

The median is the middle value of a sorted dataset.

- If n is **odd**: the median is the middle value
- If n is **even**: the median is the average of the two middle values

```
median = np.median(sales_array)
print(median)
```

Output:

```
16150.0
```

With six values sorted as `[13850, 14200, 15900, 16400, 17200, 18100]`, the median is the average of the 3rd and 4th values: $(15900 + 16400) / 2 = 16150$.

The median ignores the actual magnitude of values above and below it. This makes it **resistant to outliers**. The median only shifts when the rank order of values changes, not their size.

The Mode

The mode is the value that occurs most frequently.

```
from scipy import stats
ratings = np.array([3, 4, 4, 5, 3, 4, 2, 5, 4, 3])
result = stats.mode(ratings, keepdims=True)
print(result)
```

Output:

```
ModeResult(mode=array([4]), count=array([4]))
```

The mode is the only measure of central tendency valid for **nominal** data. You can find the most common product category, the most frequent payment method, or the most common sales region. The mean and median of those categories would be meaningless.

A dataset can have more than one mode (bimodal or multimodal) or none at all if every value occurs once. SciPy's `mode()` returns the smallest of the most frequent values when ties exist; if you need all modes, the more general `Counter` from Chapter 3 is the better tool.

2.8 Choosing the Right Measure

Situation	Best Measure	Reason
Symmetric data, no outliers	Mean	Uses all data, most precise

Situation	Best Measure	Reason
Skewed data or outliers present	Median	Not distorted by extreme values
Nominal or categorical data	Mode	Only valid option
Reporting typical salary or house price	Median	Outliers (executives, luxury properties) distort the mean

The Outlier Problem

Consider two sets of monthly sales figures:

```
import numpy as np
normal = np.array([14200, 13850, 16400, 15900, 17200, 18100])
outlier = np.array([14200, 13850, 16400, 15900, 17200, 95000])

print("Normal — mean:", np.mean(normal), " median:", np.median(normal))
print("Outlier — mean:", np.mean(outlier), " median:", np.median(outlier))
```

Output:

```
Normal — mean: 15941.666666666666 median: 16150.0
Outlier — mean: 28758.333333333332 median: 16150.0
```

The outlier nearly doubles the mean. The median does not move at all in this case, because the rank order of the middle two values is unchanged — only their size would be different, and the median is indifferent to size beyond rank.

This is why news reports on income and housing prices typically use **median** rather than mean. A small number of very high values would otherwise give a misleading picture of what is typical.

Mini Case Study — Reporting "Typical" Salary

A company has 50 employees. Forty-eight earn between 55,000 and 95,000. The CEO earns 1,200,000. The CFO earns 850,000.

A junior HR analyst computes the average salary at 112,000 and writes a recruiting brochure stating the company's "typical" salary is over 100,000. A potential hire takes the job, expecting that compensation, and is shocked when their offer comes in at \$72,000.

The mean was technically correct but practically dishonest. The median salary in this dataset would be roughly \$74,000 — far closer to what a new hire would actually receive. Whenever a small fraction of values is dramatically larger than the rest, the median is the more honest summary. Whenever you see a "typical" or "average" figure used in a report, ask whether it is the mean or the median, and whether the choice is a fair representation of what the data actually looks like.

2.9 Chapter Summary

Concept	Key Point
List	Ordered collection, indexed from 0, supports mixed types
Indexing	<code>list[0]</code> = first, <code>list[-1]</code> = last
Slicing	<code>list[start:stop]</code> — stop is excluded
NumPy array	Numerical collection with element-wise arithmetic and statistical functions
Vectorisation	One operation applied to a whole array — replaces a loop
Population vs sample	Population = all cases (N, μ); Sample = subset ($n, \bar{x}$)
Mean	Sum / count — sensitive to outliers
Median	Middle value — resistant to outliers
Mode	Most frequent value — valid for nominal data

Review Questions

1. What is the difference between `monthly_sales[3]` and `monthly_sales[3:6]` ? What does each return?
2. A dataset contains annual revenues for 500 companies, including 5 companies worth over \$10 billion. Would you report the mean or median revenue as the "typical" company revenue? Justify your answer.
3. A list contains 9 values. After sorting, the values are: `[12, 15, 18, 21, 24, 27, 30, 33, 36]` . Calculate the median by hand, then verify using `np.median()` .
4. Why can you not calculate a meaningful mean for the variable `sales_region` even if you encoded the regions as numbers (1 = Northeast, 2 = West, 3 = South)?
5. You have a NumPy array of daily sales figures for a year (365 values). Write a single line of code that increases every value by 5% and stores the result in a new variable called `projected_sales` .

Chapter 3 — Conditionals, Loops, and Frequency Distributions

The first two chapters covered values and collections of values. This chapter covers control — the ability to make decisions and to repeat work. Together they unlock most of what people mean when they say "programming." Combined with frequency distributions, they form the toolkit for the most common task in early data analysis: classifying records into categories and counting them.

3.1 Making Decisions with Conditionals

Most business logic involves decisions: flag a transaction above a threshold, classify a customer by spending tier, mark a month as above or below target, route a support ticket based on the customer's plan, decide whether a record looks suspicious enough to investigate. In Python, these decisions are handled by **conditional statements**.

```
if condition:
    # runs when condition is True
elif other_condition:
    # runs when other_condition is True
else:
    # runs when no condition above is True
```

Indentation is not optional in Python — it defines what code belongs inside each block. Consistent use of four spaces is the standard. Mixing tabs and spaces, or using two spaces in some places and four in others, will produce confusing errors. Most editors handle this for you; if you are typing into a plain text editor, configure it to insert four spaces when you press Tab.

A conditional is conceptually a sieve. The data flows in at the top, and Python passes it through each test in order. The first test that succeeds wins, and the corresponding block runs. After that, every later branch is skipped, even if its condition would also be true. Order of conditions matters.

Comparison Operators

Operator	Meaning	Example
<code>==</code>	Equal to	<code>sales == target</code>
<code>!=</code>	Not equal to	<code>region != "West"</code>
<code>></code>	Greater than	<code>sales > 10000</code>
<code><</code>	Less than	<code>sales < 10000</code>
<code>>=</code>	Greater than or equal	<code>rating >= 4</code>
<code><=</code>	Less than or equal	<code>discount <= 0.20</code>

Note the difference between `=` (assignment, stores a value) and `==` (comparison, tests equality). Confusing the two is one of the most common beginner errors. Python will raise a `SyntaxError` if you write `if x = 5:`, which is at least diagnosable; some other languages silently accept the assignment and continue with hard-to-find bugs.

Logical Operators

Multiple conditions can be combined using `and`, `or`, and `not`.

```
if sales > 10000 and region == "Northeast":
    print("High-performing Northeast sale")

if sales < 1000 or is_returned == True:
    print("Flag for review")
```

`and` requires both conditions to be true. `or` requires at least one to be true. `not` inverts a condition: `not is_returned` is true when `is_returned` is false.

A small style note: `if is_returned == True` is logically correct but verbose. Booleans can be tested directly, so `if is_returned:` is preferred. Likewise, `if not is_returned:` is cleaner than `if is_returned == False:`.

Worked Example — Performance Tiering

```
monthly_sales = 17200
target        = 15000

if monthly_sales >= target * 1.10:
    print("Exceeded target by 10% or more")
elif monthly_sales >= target:
    print("Met target")
else:
    print("Below target")
```

Output:

```
Exceeded target by 10% or more
```

Python evaluates each condition in order, top to bottom. As soon as one condition is true, it runs that block and skips the rest. Order matters — place the most specific conditions first.

If the first two conditions were swapped, every value at or above target would be classified as "Met target," and the "Exceeded by 10%" block would never run. The conditions must be ordered from strictest to loosest.

3.2 For Loops

A **for loop** repeats a block of code once for each item in a collection. This is how you process every row in a dataset, every month in a time series, or every item in a list.

```
for item in collection:
    # code that runs once per item
```

The variable name `item` is yours to choose — write it to describe what one element represents. Looping over a list of monthly sales? Use `for sales in monthly_sales:`. Looping over a list of customer names? `for customer in customers:`. Names that read like English make the code self-documenting.

Worked Example — Reporting Each Month

```
months        = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
monthly_sales = [14200, 13850, 16400, 15900, 17200, 18100]

for i, sales in enumerate(monthly_sales):
    print(f"{months[i]}: ${sales:,}")
```

Output:

```
Jan: $14,200
Feb: $13,850
Mar: $16,400
Apr: $15,900
May: $17,200
Jun: $18,100
```

Iterating with Index — `range()` and `enumerate()`

Sometimes you need both the item and its position. Two approaches:

```
# Using range(len(...)) – explicit index
for i in range(len(monthly_sales)):
    print(i, monthly_sales[i])

# Using enumerate() – cleaner, preferred
for i, sales in enumerate(monthly_sales):
    print(i, sales)
```

`enumerate()` is preferred in practice because it is more readable and less error-prone. It also tends to be slightly faster.

Accumulating Results

A common pattern is to start with an empty list and append results inside the loop.

```
target = 15000
above_target = []
for i, sales in enumerate(monthly_sales):
    if sales >= target:
        above_target.append(months[i])

print(above_target)
```

Output:

```
['Mar', 'Apr', 'May', 'Jun']
```

This pattern — initialise, loop, accumulate — appears constantly in data processing. Once you recognise it, you will see it everywhere. The next section shows how to write the same idea in a single line using a list comprehension.

3.3 List Comprehensions

A **list comprehension** is a concise way to create a new list by applying an expression to every item in a collection. It compresses a loop into a single line.

```
# Loop version
projected = []
for sales in monthly_sales:
    projected.append(sales * 1.05)

# Comprehension version – same result
projected = [sales * 1.05 for sales in monthly_sales]
```

The two snippets produce the same result. The comprehension version is more compact and, with practice, easier to scan. The general rule of thumb: if the loop body is short and its only purpose is to build a new list, use a comprehension. If the loop body is more than a couple of lines or has side effects (like printing), use an explicit loop.

Filtering with a Condition

Add an `if` clause at the end to include only items that meet a condition.

```
above_target = [sales for sales in monthly_sales if sales >= 15000]
print(above_target)
```

Output:

```
[16400, 15900, 17200, 18100]
```

Inline if-else for Labelling

A different `if` placement — in the middle — produces one value or another based on a condition. This is useful for labelling.

```
labels = ["High" if sales >= 15000 else "Low" for sales in monthly_sales]
print(labels)
```

Output:

```
['Low', 'Low', 'High', 'High', 'High', 'High']
```

Note the position of `if` differs between filtering (`if` at the end) and labelling (`if` in the middle). These are two distinct patterns and the syntax catches everyone out at first.

Purpose	Syntax
Filter items	<code>[x for x in data if condition]</code>
Transform items	<code>[expression for x in data]</code>
Label items	<code>[a if condition else b for x in data]</code>

Comprehensions can be nested, but readability suffers quickly. If you find yourself writing a comprehension with two `for` clauses and an `if`, expand it back into a regular loop.

3.4 Frequency Distributions

Stats Connection

A **frequency distribution** shows how often each value or category appears in a dataset. It is typically the first step in understanding a categorical variable — answering the question *what is in this column?* before any other analysis.

Type	Description	Example
Frequency	Count of occurrences	Northeast: 7 transactions
Relative frequency	Proportion of total	Northeast: 35%
Cumulative frequency	Running total	Top 2 regions: 63% of transactions

Frequency distributions are most meaningful for **nominal** and **ordinal** data — region, payment method, product category, rating tier. For continuous data (revenue, temperature), you group values into bins first (covered in Week 5 with histograms).

A frequency distribution is also a sanity check. If you expect "five sales regions" but the frequency table shows seven, you have spotted a data quality issue (probably an inconsistent spelling — "northeast" vs "Northeast" — that you would otherwise have missed).

Building a Frequency Table Manually

Python dictionaries map keys to values, making them a natural structure for frequency tables.

```
regions = ["Northeast", "West", "Northeast", "South", "West",
           "Northeast", "Midwest", "Northeast", "South", "West"]

freq = {}
for region in regions:
    if region in freq:
        freq[region] += 1
    else:
        freq[region] = 1

print(freq)
```

Output:

```
{'Northeast': 4, 'West': 3, 'South': 2, 'Midwest': 1}
```

This is a foundational pattern: walk the list, look up the current count for the key, and either increment or initialise it. It is worth being able to write it from memory once or twice — but in real code, you should reach for the standard library tool below.

Using Counter

`collections.Counter` automates this in one line and adds useful methods.

```
from collections import Counter

freq = Counter(regions)
print(freq)
```

Output:

```
Counter({'Northeast': 4, 'West': 3, 'South': 2, 'Midwest': 1})
```

```
# Most common values
print(freq.most_common(2))
```

Output:

```
[('Northeast', 4), ('West', 3)]
```

`Counter` is part of the standard library — no installation needed — and is the right tool for any "how many of each thing" question.

Relative Frequency

Convert counts to proportions by dividing by the total.

```
total = len(regions)
for region, count in freq.items():
    pct = (count / total) * 100
    print(f"{region}: {count} ({pct:.1f}%")
```

Output:

```
Northeast: 4 (40.0%)
West: 3 (30.0%)
South: 2 (20.0%)
Midwest: 1 (10.0%)
```

Relative frequencies are particularly useful when comparing across datasets of different sizes. Twenty support tickets per week from a region that handles a thousand customers is very different from twenty tickets per week from a region with a hundred customers, even though the raw counts are identical.

3.5 Outlier Detection

Stats Connection

An **outlier** is a value that falls unusually far from the rest of the data. In business data, outliers are common and meaningful — they can represent exceptional sales, data entry errors, fraud, or genuine anomalies worth investigating.

Identifying outliers is not about removing them. It is about knowing they exist and deciding whether they should be included in a given analysis. Sometimes an outlier is a data problem that should be fixed. Sometimes it is

the most interesting point in the dataset — a single fraudulent transaction, a viral marketing event, a bulk order that demonstrates capacity for growth.

The 2 Standard Deviation Rule

A simple and widely used threshold: flag any value more than two standard deviations from the mean.

Outlier if: $|x - \bar{x}| > 2\sigma$

```
import numpy as np
transactions = np.array([320, 4800, 750, 12500, 980, 6200,
                        430, 8900, 1100, 275, 95000])

mean = np.mean(transactions)
std = np.std(transactions)

outliers = [x for x in transactions if abs(x - mean) > 2 * std]
print(f"Mean: {mean:.0f}, Std: {std:.0f}")
print(f"Outliers: {outliers}")
```

Output:

```
Mean: 11932, Std: 27226
Outliers: [95000]
```

`abs()` computes the absolute value — the distance from the mean regardless of direction — so both unusually high and unusually low values are caught.

Limitation of This Method

Look at the standard deviation in the output above: 27,226. The "typical" transaction in this dataset is well under 5,000, but the standard deviation is enormous because the \$95,000 outlier is dragging it up. The rule's threshold expands accordingly, and only the very largest value is flagged. Several other suspicious-looking transactions slip through.

The mean and standard deviation are themselves sensitive to outliers. A very large outlier inflates the standard deviation, widening the threshold and making it harder to flag the outlier in the first place — sometimes called the "masking" problem. In Chapter 4, we introduce a more robust method based on the **interquartile range (IQR)**, which is not affected by extreme values.

The 2 standard deviation rule is a useful starting point for symmetric, well-behaved data. For production data pipelines and especially for skewed business data, the IQR method is generally preferred.

3.6 Putting It Together — A Classification and Summary Workflow

A realistic data task often combines all three concepts from this chapter.

```

purchases = [320, 4800, 750, 12500, 980, 6200, 430, 8900, 1100, 275]

# Step 1 – Classify with a comprehension
tiers = ["Large" if x >= 5000 else "Medium" if x >= 1000 else "Small"
         for x in purchases]
print("Tiers:", tiers)

# Step 2 – Frequency distribution
from collections import Counter
tier_freq = Counter(tiers)

# Step 3 – Relative frequency
total = len(purchases)
print("\nDistribution:")
for tier, count in tier_freq.most_common():
    pct = (count / total) * 100
    print(f" {tier}: {count} ({pct:.1f}%)")

# Step 4 – Outlier check
import numpy as np
arr = np.array(purchases)
mean = np.mean(arr)
std = np.std(arr)
outliers = [x for x in purchases if abs(x - mean) > 2 * std]
print(f"\nOutliers (2 std rule): {outliers}")

```

Output:

```

Tiers: ['Small', 'Medium', 'Small', 'Large', 'Small', 'Large', 'Small', 'Large', 'Medium', 'Small']

Distribution:
  Small: 5 (50.0%)
  Large: 3 (30.0%)
  Medium: 2 (20.0%)

Outliers (2 std rule): []

```

This four-step pattern — classify, count, proportion, flag — is the foundation of exploratory data analysis (EDA), which we build on in Week 5 and 6.

Mini Case Study — Customer Segmentation

A regional bank wants to segment its 50,000 small business customers by monthly transaction volume to tailor service offerings. The raw data is a single column of monthly transaction totals. The analyst's first pass:

```

# Pseudo-code style, full data not shown
tiers = ["Premium" if x >= 100000 else
         "Standard" if x >= 20000 else
         "Basic"
         for x in monthly_volumes]

tier_counts = Counter(tiers)
print(tier_counts.most_common())

```

The output reveals roughly 70% Basic, 25% Standard, 5% Premium. That distribution drives concrete business decisions: dedicated relationship managers for Premium clients, automated digital servicing for Basic, and a hybrid model for Standard. The same chapter-level tools — comprehensions for classification, `Counter` for distribution — scale from a handful of values to fifty thousand without changing approach.

3.7 Chapter Summary

Concept	Key Point
<code>if / elif / else</code>	Evaluates conditions top-to-bottom, runs first true block
<code>==</code> vs <code>=</code>	<code>==</code> compares, <code>=</code> assigns — do not confuse them
<code>and / or</code>	Combine conditions; <code>and</code> requires both true, <code>or</code> requires one
<code>for</code> loop	Repeats for every item in a collection
<code>enumerate()</code>	Returns index and value together — preferred over <code>range(len(...))</code>
List comprehension	<code>[expr for x in data if cond]</code> — compact loop in one line
Frequency distribution	Count of each category — first step for nominal/ordinal data
<code>Counter</code>	Builds a frequency table in one line with <code>.most_common()</code>
Outlier (2 std rule)	Flag values where <code>abs(x - mean) > 2 * std</code>
Masking	Outliers inflate std and can hide other outliers

Review Questions

1. What is wrong with the following code? How would you fix it?

```
if sales = 15000:
    print("On target")
```

2. A list contains 100 transaction amounts. Write a single list comprehension that returns only the transactions above \$5,000, with each value increased by 8%.
3. Explain in plain language why the 2 standard deviation outlier rule can underperform when the data contains a very large outlier.
4. You have a list of product categories from 200 orders. Write the code to build a frequency table and print each category with its count and percentage share, sorted from most to least common.
5. The `Counter` method `.most_common(n)` returns the top `n` items. What does `.most_common(1)[0][0]` return? Walk through each step of that expression.

Chapter 4 — Functions, Error Handling, and an Introduction to Pandas

This is the central chapter of the course. It introduces functions, which are the primary tool for keeping code organised as projects grow; error handling, which is what separates a fragile script from a robust one; and Pandas, which is the library that turns Python into a serious data analysis tool. On the statistics side, it introduces the measures of spread — variance, standard deviation, z-scores, and the IQR — that move you from describing the centre of a dataset to describing its shape.

4.1 Why Functions Matter

By Week 3, you were writing the same calculations repeatedly — mean, percentage change, outlier detection. Functions eliminate that repetition. Write the logic once, give it a name, and call it whenever you need it.

Beyond convenience, functions make code easier to read, test, and fix. When a calculation is wrong, you fix it in one place instead of hunting through every cell where you copy-pasted it. When a calculation needs to change — say, the company switches from a 1.5x to a 1.75x discount threshold — you update one line, and every caller automatically uses the new logic.

Functions also encourage *thinking in inputs and outputs*. A well-written function takes some inputs, produces an output, and changes nothing else about the world. This is a discipline that pays off when datasets get large and bugs get subtle. Code that "just changes a global variable somewhere" is famously hard to debug; code that returns a clean output for a given input is much easier to reason about.

4.2 Defining and Calling Functions

```
def function_name(parameter1, parameter2):  
    # code  
    return result
```

- `def` declares the function
- **Parameters** are the inputs the function expects
- `return` sends a value back to the caller — without it, the function returns `None`

```
def pct_change(old, new):  
    change = ((new - old) / old) * 100  
    return change  
  
print(pct_change(48000, 54600))    # 13.75  
print(pct_change(15000, 13500))    # -10.0
```

Output:

```
13.75  
-10.0
```

The function is defined once and called twice with different arguments. The logic lives in one place. Compare this with the alternative — typing `((54600 - 48000) / 48000) * 100` in one place and `((13500 - 15000) / 15000) * 100` in another. Either of those formulas could have a typo; only the function definition is checked once and reused with confidence.

Parameters vs Arguments

The terminology is small but worth getting right. **Parameters** are the names in the function definition — `old` and `new` above. **Arguments** are the actual values passed at the call site — `48000` and `54600`. The two terms are often used interchangeably in conversation but have distinct meanings in technical writing.

Default Parameters

Parameters can have **default values**. If the caller omits that argument, the default is used.

```
def project_revenue(revenue, growth_rate=0.05):  
    return revenue * (1 + growth_rate)  
  
print(project_revenue(100000))          # uses 5% default  
print(project_revenue(100000, 0.12))    # uses 12%
```

Output:

```
105000.0  
112000.0
```

Default parameters should always come after non-default ones in the function signature. They are most useful for parameters that have a sensible "usual" value but might occasionally need to be overridden.

Returning Multiple Values

A function can return more than one value. Python packs them into a tuple, which can be unpacked on the receiving end.

```
import numpy as np  
  
def summary_stats(data):  
    return np.mean(data), np.median(data), np.std(data, ddof=1)  
  
sales = [14200, 13850, 16400, 15900, 17200, 18100]  
mean, median, std = summary_stats(sales)  
print(f"Mean: {mean:.0f}, Median: {median:.0f}, Std: {std:.0f}")
```

Output:

```
Mean: 15942, Median: 16150, Std: 1626
```

This is a clean way to package up "everything you usually want about a dataset" into a single call.

Docstrings

A short description placed at the start of a function — between triple quotes — is called a docstring. It documents what the function does and is shown when someone calls `help(function_name)`.

```
def pct_change(old, new):  
    """Return the percentage change from old to new value."""  
    return ((new - old) / old) * 100
```

In a small project the docstring may seem like overkill. In a project with twenty functions, it is what saves you when you come back to your code three months later.

4.3 Error Handling

Functions can receive unexpected inputs — an empty list, a string where a number was expected, a zero where a denominator was assumed to be non-zero. Unhandled, these cause crashes. `try / except` lets you handle them gracefully.

```
try:  
    # code that might fail  
except ErrorType:  
    # what to do if it fails
```

```
def safe_pct_change(old, new):  
    try:  
        return round(((new - old) / old) * 100, 2)  
    except ZeroDivisionError:  
        return "Error: base value cannot be zero"  
  
print(safe_pct_change(48000, 54600))  
print(safe_pct_change(0, 1000))
```

Output:

```
13.75  
Error: base value cannot be zero
```

You can handle multiple error types in sequence.

```
def safe_mean(data):  
    try:  
        return sum(data) / len(data)  
    except TypeError:  
        return "Error: data must be a list of numbers"  
    except ZeroDivisionError:  
        return "Error: list is empty"
```

Common Error Types

Error	Cause
<code>ZeroDivisionError</code>	Dividing by zero
<code>TypeError</code>	Wrong data type (e.g., adding a string to a number)
<code>ValueError</code>	Right type, wrong value (e.g., <code>int("abc")</code>)
<code>IndexError</code>	Accessing a list index that doesn't exist
<code>KeyError</code>	Accessing a dictionary key that doesn't exist

Error handling is not about catching every possible failure — it is about anticipating the ones that are likely given your data and use case. A bare `except:` that swallows every error is almost always a mistake; it hides bugs and makes them harder to fix later. Catch the specific errors you expect and let the unexpected ones bubble up loudly.

4.4 Introduction to Pandas

NumPy arrays are powerful for numerical computation, but they have no column names, no row labels, and no support for mixed data types in the same structure. Real datasets have all of these.

Pandas provides two structures designed for real-world tabular data. Pandas is to data analysis what spreadsheets are to office work — its core abstractions match the way analysts already think about data, but they scale to millions of rows and integrate cleanly with the rest of the Python ecosystem.

Series

A `Series` is a single column of data with an index. Think of it as a labelled NumPy array.

```
import pandas as pd

monthly = pd.Series([14200, 13850, 16400, 15900, 17200, 18100],
                    index=["Jan", "Feb", "Mar", "Apr", "May", "Jun"])
print(monthly)
print(monthly["Mar"])
```

Output:

```
Jan    14200
Feb    13850
Mar    16400
Apr    15900
May    17200
Jun    18100
dtype: int64
16400
```

Notice that the index is preserved on every operation — you can multiply the whole `Series` by 1.10 and the labels travel along, so the result still tells you which month is which.

DataFrame

A `DataFrame` is a table of rows and columns — the Pandas equivalent of a spreadsheet. Each column is a Series.

```
data = {
    "rep":      ["Chen", "Patel", "Okafor", "Rivera", "Thompson"],
    "region":   ["Northeast", "West", "South", "Midwest", "Northeast"],
    "q1_sales": [42100, 38900, 45300, 31800, 39200],
    "q2_sales": [44500, 41200, 43800, 35600, 42100]
}

df = pd.DataFrame(data)
print(df)
```

Output:

	rep	region	q1_sales	q2_sales
0	Chen	Northeast	42100	44500
1	Patel	West	38900	41200
2	Okafor	South	45300	43800
3	Rivera	Midwest	31800	35600
4	Thompson	Northeast	39200	42100

A `DataFrame` stores its columns as separate arrays under the hood, but it presents them as a unified table. You can access them as columns, slice them as rows, filter on conditions, group, pivot, and visualise — all with a consistent syntax.

Exploring a DataFrame

Method / Attribute	What It Returns
<code>df.shape</code>	(rows, columns) as a tuple
<code>df.dtypes</code>	Data type of each column
<code>df.head(n)</code>	First <code>n</code> rows (default 5)
<code>df.tail(n)</code>	Last <code>n</code> rows
<code>df.info()</code>	Column names, types, and non-null counts
<code>df.describe()</code>	Summary statistics for numeric columns

Running these on a fresh `DataFrame` is the standard "first contact" routine. It tells you how big the dataset is, what is in it, and where the obvious problems are.

Accessing Data

```
# Single column – returns a Series
df["q1_sales"]

# Multiple columns – returns a DataFrame
df[["rep", "q1_sales"]]

# Row by position
df.iloc[0]          # first row

# Row by label/condition
df.loc[df["region"] == "Northeast"]
```

`.iloc[]` ("integer location") accesses by row position. `.loc[]` ("location") accesses by label or by a boolean condition. Beginners sometimes confuse the two. The rule of thumb: if you are using a number, use `.iloc[]`; if you are using a name or a condition, use `.loc[]`.

Adding Calculated Columns

Pandas column arithmetic is element-wise, just like NumPy.

```
df["h1_total"] = df["q1_sales"] + df["q2_sales"]
df["q2_growth_pct"] = ((df["q2_sales"] - df["q1_sales"]) / df["q1_sales"]) * 100
print(df)
```

Output:

	rep	region	q1_sales	q2_sales	h1_total	q2_growth_pct
0	Chen	Northeast	42100	44500	86600	5.700713
1	Patel	West	38900	41200	80100	5.913882
2	Okafor	South	45300	43800	89100	-3.311258
3	Rivera	Midwest	31800	35600	67400	11.949686
4	Thompson	Northeast	39200	42100	81300	7.397959

This is one of Pandas' most useful features — adding a derived column is one line, and it operates on the whole column at once. The same logic in raw Python would require an explicit loop or a comprehension; in Pandas it is a single readable expression.

4.5 Variance and Standard Deviation

The mean tells you the centre. **Variance** and **standard deviation** tell you the spread — how far values typically sit from that centre.

Why Spread Matters

Two sales reps can have the same mean monthly revenue but very different reliability. Rep A consistently hits close to target. Rep B swings wildly — some months exceptional, some months poor. The mean obscures this. Spread measures reveal it.

A sales manager who only looks at means will rate the two reps as equally productive. A manager who also looks at spread will see that Rep A is dependable and Rep B is volatile, and will plan, coach, and forecast differently for each.

Variance

Variance is the average squared deviation from the mean.

Sample variance (use this when working with a sample, which is almost always):

$$s^2 = (\sum (x - \bar{x})^2) / (n-1)$$

The squaring serves two purposes: it makes negative deviations contribute the same as positive ones (otherwise they would cancel out), and it gives extra weight to large deviations. Both are usually what you want.

The $n-1$ denominator (Bessel's correction) adjusts for the fact that a sample tends to underestimate the true population spread. The intuition: when computing deviations from the *sample* mean rather than the unknown true mean, the deviations are slightly too small. Dividing by $n-1$ instead of n compensates. For large n , the correction is negligible; for small n , it matters.

Standard Deviation

Standard deviation is the square root of variance. This brings the units back to the original scale — dollars, not dollars squared.

$$s = \sqrt{s^2}$$

```
import numpy as np
rep_a = np.array([15200, 15800, 14900, 16100, 15400, 15600])
rep_b = np.array([9800, 21000, 12500, 18900, 11200, 19600])

print(f"Rep A: mean={np.mean(rep_a):.0f}, std={np.std(rep_a, ddof=1):.0f}")
print(f"Rep B: mean={np.mean(rep_b):.0f}, std={np.std(rep_b, ddof=1):.0f}")
```

Output:

```
Rep A: mean=15500, std=434
Rep B: mean=15500, std=4727
```

Both reps average exactly 15,500. The standard deviation tells the real story — Rep A's monthly sales fluctuate by about 434 around their average, while Rep B's swing by nearly \$4,700. Same mean, very different operational reality.

ddof=1 : NumPy's `np.std()` defaults to population standard deviation (`ddof=0`). Always pass `ddof=1` for sample data. Pandas' `.std()` defaults to `ddof=1` — the correct choice. This is one of the most common silent bugs in early Python data work.

4.6 Z-Scores

A **z-score** expresses a value as its distance from the mean, measured in standard deviations.

$$z = (x - \bar{x}) / (s)$$

Z-score	Interpretation
0	Exactly at the mean
+1.5	1.5 standard deviations above the mean
-2.1	2.1 standard deviations below the mean
z	z

Uses of Z-Scores

Comparing across scales. A rep in a high-volume region and a rep in a low-volume region cannot be compared by raw sales. Z-scores normalise both to the same scale, making comparison fair. A rep who is +1.5 standard deviations above their region's mean is performing well, regardless of whether that region's mean is 30,000 or 80,000.

Outlier detection. Values with $|z| > 2$ (or sometimes > 3) are flagged for review.

Standardisation for further analysis. Many statistical models — clustering, regression with regularisation, principal components analysis — work better when the input variables are on the same scale. Z-scoring is the standard way to put them there.

```
import numpy as np
regional_sales = np.array([42100, 38900, 45300, 31800, 39200, 44500, 41200])
z_scores = (regional_sales - np.mean(regional_sales)) / np.std(regional_sales, ddof=1)
print(z_scores.round(2))
```

Output:

```
[ 0.36 -0.36  1.07 -1.94 -0.29  0.91  0.24]
```

The value with $z = -1.94$ is nearly two standard deviations below the mean. By the conventional rule, it would warrant a closer look — not necessarily because it is wrong, but because something distinguishes that month or region from the rest.

4.7 The Interquartile Range and Robust Outlier Detection

Quartiles and the IQR

Percentiles divide a sorted dataset into 100 equal parts. The key ones:

- **Q1 (25th percentile):** 25% of values fall below this point
- **Q2 (50th percentile):** the median
- **Q3 (75th percentile):** 75% of values fall below this point

The **interquartile range** is the distance between Q1 and Q3:

$$\text{IQR} = Q3 - Q1$$

The IQR covers the middle 50% of the data. It is not affected by extreme values in the tails — by construction, the top and bottom 25% of values can be anything at all without changing Q1, Q3, or the IQR. This makes the IQR a robust measure of spread.

The Tukey Fence Rule

The standard IQR-based outlier detection rule (Tukey fences, after the statistician John Tukey who introduced it) flags values outside:

Lower fence = $Q1 - 1.5 \times IQR$ Upper fence = $Q3 + 1.5 \times IQR$

```
import numpy as np
data = np.array([320, 4800, 750, 12500, 980, 6200,
                 430, 8900, 1100, 275, 95000])

q1      = np.percentile(data, 25)
q3      = np.percentile(data, 75)
iqr     = q3 - q1
lower_fence = q1 - 1.5 * iqr
upper_fence = q3 + 1.5 * iqr

outliers = data[(data < lower_fence) | (data > upper_fence)]
print(f"Q1: {q1}, Q3: {q3}, IQR: {iqr}")
print(f"Fences: [{lower_fence:.0f}, {upper_fence:.0f}]")
print(f"Outliers: {outliers}")
```

Output:

```
Q1: 540.0, Q3: 7550.0, IQR: 7010.0
Fences: [-9975, 18065]
Outliers: [95000]
```

IQR vs Standard Deviation Rule

Method	Based On	Affected by Outliers?	Best Used When
2 std rule	Mean, std	Yes	Roughly symmetric data
IQR (Tukey)	Quartiles	No	Skewed data or when outliers are already suspected

In business data — where a single bulk order or executive salary can skew a dataset significantly — the IQR method is generally more reliable. It is also the default rule used by boxplots in matplotlib, seaborn, and almost every other plotting library, so adopting it makes your numerical analysis line up with what you see visually.

Mini Case Study — Dirty Data, Clean Reporting

A retailer reports daily revenue from 200 stores. One day, a single store reports 9.4 million in revenue — three orders of magnitude above the typical store's 9,000 daily revenue. The cause turns out to be an integration bug

duplicating a transaction many times.

If the analyst had reported the day using the mean, the daily total would have looked like a record-breaking surge. The mean would have been pulled up by roughly 47,000 — almost a doubling — by a single bogus row. Using the median instead, the daily figure barely moves: the bad row becomes one rank among 200, and that rank is at the top regardless of whether the value is 9,400,000 or \$9,400.

The analyst's actual workflow used the IQR-based outlier check on the day's 200 store totals. The bad row was flagged immediately, the engineering team was notified, and the executive summary went out using a corrected number. This is the practical payoff for using robust statistics: they protect your reporting from the noise that real data systems produce.

4.8 Chapter Summary

Concept	Key Point
Function	<code>def name(params): return value</code> — write once, call anywhere
Default parameter	Provides a fallback value when the argument is omitted
Docstring	Triple-quoted description at the start of a function
<code>try / except</code>	Handles anticipated errors without crashing
Pandas Series	A single labelled column of data
Pandas DataFrame	A labelled table — rows, columns, mixed types
<code>.iloc[]</code> vs <code>.loc[]</code>	Position-based vs label/condition-based indexing
<code>ddof=1</code>	Use for sample standard deviation in NumPy
Variance	Average squared deviation — measures spread
Standard deviation	Square root of variance — same units as data
Z-score	Deviation from mean in std units — enables comparison and outlier detection
IQR	Spread of middle 50% — not affected by extreme values
Tukey fences	$Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$

Review Questions

1. What is the difference between a **parameter** and an **argument**? Give an example of each.
2. A function `calculate_growth(revenue, rate)` is called as `calculate_growth(50000)`. What happens, and how would you fix the function definition to allow this call to work with a default rate of 3%?
3. Two datasets have the same mean of 42,000. Dataset A has a standard deviation of 1,200. Dataset B has a standard deviation of \$9,800. What does this tell you about each dataset?

4. A sales figure has a z-score of +2.4. What does this mean in plain language? Would you flag it as an outlier?
5. Explain in your own words why the IQR-based outlier rule is more robust than the 2 standard deviation rule when the data already contains outliers.

Chapter 5 — Data Cleaning and Visualisation

The first four chapters operated on data that arrived in clean shape. Real datasets do not. This chapter covers the work that comes between *getting* data and *analysing* it — identifying problems, deciding what to do about them, and using visualisation both to spot issues and to communicate findings.

5.1 Why Data Cleaning Comes First

Raw data is rarely analysis-ready. Before drawing conclusions, you need to know what you are actually working with. Missing values, invalid entries, and inconsistent formats all produce misleading results if left unaddressed.

Industry estimates of how much time data analysts spend on cleaning vary, but a figure between 60% and 80% comes up consistently in surveys. This sounds excessive until you have lived through it. A model trained on dirty data is worse than no model. A dashboard fed by dirty data leads stakeholders to wrong decisions and erodes trust when the errors come to light.

Data cleaning is not a preliminary chore — it is part of the analysis. Decisions made during cleaning (what to fill, what to drop, what to flag) directly shape what the data says. Two analysts cleaning the same dataset can reach different reasonable conclusions, and the right answer depends on the business context, not on a universal rule.

A useful rule of thumb: cleaning decisions should be defensible. You should be able to explain to a stakeholder, in plain language, what you did and why. If a decision would be uncomfortable to defend, it probably should not be made.

5.2 Loading Data

In real work, data comes from files. The most common format is CSV (comma-separated values).

```
import pandas as pd

df = pd.read_csv("quarterly_sales.csv")
```

Pandas reads many other formats out of the box: `read_excel` for `.xlsx` files, `read_json` for JSON, `read_sql` for database queries, `read_parquet` for the columnar format used in modern data warehouses. The interface is consistent — read in, analyse, optionally write back out with `to_csv`, `to_excel`, etc.

Once loaded, your first task is to understand what you have.

```
df.shape      # (rows, columns)
df.dtypes     # data type of each column
df.head()     # first 5 rows
df.info()     # column names, types, non-null counts
df.describe() # summary statistics for numeric columns
```

`df.describe()` returns count, mean, std, min, quartiles, and max for every numeric column. It is often the first statistical summary you run on a new dataset, and it is surprisingly informative. A negative minimum on a `price` column, for example, immediately reveals a data problem you might otherwise miss for hours.

5.3 Missing Data

Identifying Missing Values

```
df.isnull().sum()          # count per column
df.isnull().sum() / len(df) * 100 # as percentage
```

The first thing to learn about a missing value is *why* it is missing. Missing because the customer declined to answer the survey question is different from missing because a sensor failed, which is different again from missing because the database column did not exist before last month. The reason determines the right strategy.

Strategies for Handling Missing Data

Strategy	When to Use
Drop rows	Few missing values, missing at random
Fill with median	Numeric data, distribution may be skewed
Fill with mean	Numeric data, distribution is symmetric
Fill with mode	Categorical data
Fill with a constant	When a specific value (e.g., 0) makes business sense
Flag and investigate	When the missingness itself may be meaningful

```
# Fill missing revenue with the median
df["revenue"] = df["revenue"].fillna(df["revenue"].median())

# Drop rows where any value is missing
df.dropna(inplace=True)
```

Why median over mean for imputation? If the distribution is skewed — for example, a few very large transactions in a sales dataset — the mean is pulled toward the high end. Filling missing values with the mean implicitly assumes the missing observations were above-average, which may not be true. The median is more robust.

Why not fill with zero? Zero means something: no revenue, no units sold. If a transaction occurred but its value is unknown, filling with zero misrepresents it as a transaction with zero value. This is a common mistake that quietly distorts analysis. The same applies to any other "magic" fill value — `-1`, `-999`, or whatever convention an upstream system used. The right approach is usually to convert those to genuine `NaN` values first and then handle them deliberately.

When Missingness Itself Is the Signal

In some datasets, *whether* a value is missing is more interesting than what it would have been. A missing `discount_pct` might indicate the customer was a contract account where discounts are negotiated separately. A missing `delivery_address` on an online order might indicate a digital product. In both cases, blindly filling the missing values would erase real information about the underlying business process.

A defensive practice: before imputing, add a column called `was_<colname>_missing` that records the original missingness as a boolean. You can drop it later if it turns out not to matter; you cannot recover it once it is gone.

5.4 Data Quality

Missing values are only one category of data problem. Values can also be present but wrong.

Common Quality Issues in Business Data

Issue	Example
Negative values in non-negative fields	<code>revenue = -450</code>
Out-of-range values	<code>discount_pct = 1.50</code> (150% discount)
Impossible combinations	<code>units_sold = 0</code> but <code>revenue = 12000</code>
Duplicate rows	Same transaction recorded twice
Inconsistent capitalisation	<code>"Northeast"</code> and <code>"northeast"</code> as different values
Typo variants	<code>"Cred Card"</code> instead of <code>"Credit Card"</code>
Trailing whitespace	<code>"Northeast "</code> does not equal <code>"Northeast"</code>

Finding and Fixing Invalid Values

```
# Find rows with invalid discounts
invalid = df[df["discount_pct"] < 0]

# Replace with 0
df.loc[df["discount_pct"] < 0, "discount_pct"] = 0

# Check for duplicates
df.duplicated().sum()

# Remove duplicates
df.drop_duplicates(inplace=True)
```

The `df.loc[condition, column] = value` pattern is the standard way to set values conditionally in Pandas. It works on a Boolean mask — `df["discount_pct"] < 0` is a Series of `True / False` values — and applies the assignment only to the rows where the mask is true.

Standardising Categorical Values

```
# Strip whitespace and standardise capitalisation
df["region"] = df["region"].str.strip().str.title()
```

Pandas' `.str` accessor exposes string methods on a column. `strip()` removes leading and trailing whitespace; `title()` capitalises the first letter of each word. After applying both, `"Northeast "`, `"northeast"`, and `"NORTHEAST "` all collapse to `"Northeast"`, and grouping behaves the way you expect.

Safe Division with `np.where`

When adding a calculated column that involves division, guard against zero denominators.

```
import numpy as np

df["revenue_per_unit"] = np.where(
    df["units_sold"] > 0,
    df["revenue"] / df["units_sold"],
    np.nan
)
```

`np.where(condition, value_if_true, value_if_false)` is a vectorised if-else — it operates on the entire column at once. Without this guard, dividing by zero would either crash the script or produce `inf` values that quietly propagate into your reports.

Mini Case Study — Cleaning Before Reporting

A consumer goods company runs a weekly sales report by region. One Monday, the report shows the West region jumping from 1.2M to 1.8M in a single week — a 50% spike with no marketing event behind it. The data engineer investigating finds three issues in the source file:

1. Two duplicate transactions, totalling \$46,000, that the upstream system inserted twice during a retry.
2. One row with `region = "West "` (trailing space) that previously had been counted as a separate region and is now being grouped correctly with `"West"`, inflating the total.
3. Five rows with negative discount percentages, each of which had inflated reported revenue.

After deduplicating, trimming whitespace, and clipping invalid discounts, the West region's actual week-over-week change is roughly +4%. The original "spike" was entirely a data quality artefact. An analyst who had skipped the cleaning step would have spent the week chasing a non-existent marketing success story.

5.5 Visualisation with Matplotlib and Seaborn

Python has two main visualisation libraries:

Library	Role
Matplotlib	Low-level, full control, verbose syntax
Seaborn	Built on Matplotlib, simpler syntax for statistical plots

Most analysts use both — Seaborn for quick, clean statistical charts; Matplotlib for fine-grained customisation. A typical workflow uses Seaborn to draft the chart and Matplotlib to tweak titles, axis labels, and styling for the final version.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")    # clean default style
```

A chart serves one of two purposes: *exploration* (helping you understand the data) or *communication* (helping someone else understand your findings). Exploration charts can be quick and ugly — they are for you. Communication charts deserve more time on labels, titles, and colour choices, because they will travel beyond your screen.

5.6 Histograms

A **histogram** bins a continuous variable and plots frequency per bin. It reveals the **shape** of the distribution.

```
# Matplotlib
plt.hist(df["revenue"], bins=20, color="steelblue", edgecolor="white")
plt.axvline(df["revenue"].mean(), color="red", linestyle="--", label="Mean")
plt.axvline(df["revenue"].median(), color="orange", linestyle="--", label="Median")
plt.legend()
plt.show()

# Seaborn with KDE overlay
sns.histplot(df["revenue"], bins=20, kde=True)
plt.show()
```

The `bins` parameter controls how many bars the chart uses. Too few bins hide structure; too many produce a jagged plot dominated by random fluctuations. There is no universal best choice; 20 to 30 bins is a sensible default for most business datasets.

Distribution Shapes

Shape	Description	Mean vs Median
Symmetric	Mirror image left and right	Mean $\approx$ Median
Right-skewed	Long tail to the right	Mean $>$ Median
Left-skewed	Long tail to the left	Mean $<$ Median

Shape	Description	Mean vs Median
Bimodal	Two peaks	Two sub-populations likely present

Most business data — sales, revenue, durations, file sizes — is right-skewed. A few large transactions stretch the right tail, pulling the mean above the median. The mean-median gap is a quick numerical proxy for skewness.

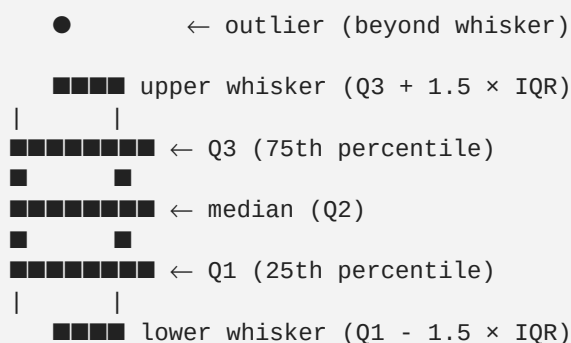
The KDE (kernel density estimate) is a smoothed curve that makes the shape easier to read than raw bars, especially with smaller datasets. It is often shown alongside the histogram or instead of it.

Bimodality Is Often a Story

If you plot a histogram and it has two distinct peaks, that is rarely random. It usually means the dataset contains two underlying populations that should be analysed separately. Customer lifetime value with two peaks might indicate "occasional buyers" and "subscribers." Order size with two peaks might indicate retail and wholesale. Spotting bimodality is one of the highest-value uses of a quick histogram.

5.7 Boxplots

A **boxplot** (box-and-whisker plot) visualises the five-number summary and highlights outliers.



Boxplots are especially useful for **comparing groups** — plotting one box per category side by side immediately shows differences in centre, spread, and outliers.

```
sns.boxplot(data=df, x="region", y="revenue", palette="Set2", hue="region", legend=False)
plt.title("Revenue by Region")
plt.show()
```

A glance at the resulting chart usually surfaces three things: which region has the highest typical revenue (compare the medians), which region is most variable (compare the box heights), and which regions have notable outliers (look at the dots beyond the whiskers). A boxplot is one of the densest visualisations available for grouped numeric data.

The whiskers extend to the most extreme value within $1.5 \times IQR$ of the box — directly mirroring the Tukey outlier rule from Chapter 4. The conceptual link is intentional: the boxplot was designed by Tukey precisely as a graphical version of his outlier criterion.

5.8 Scatter Plots

A **scatter plot** shows the relationship between two continuous variables. Each row in the DataFrame becomes one point.

```
sns.scatterplot(data=df, x="units_sold", y="revenue",  
                hue="region", alpha=0.7)  
plt.title("Units Sold vs Revenue")  
plt.show()
```

The `hue` parameter colours points by a categorical variable, adding a third dimension to the chart. The `alpha=0.7` parameter makes points slightly transparent — a small detail that prevents overplotting from hiding density patterns when many points overlap.

What to Look For

Pattern	What It Suggests
Points rising left to right	Positive relationship
Points falling left to right	Negative relationship
No clear direction	Weak or no linear relationship
Curved pattern	Non-linear relationship
Tight clustering	Strong relationship
Diffuse cloud	Weak relationship
Distinct clusters	Possibly multiple sub-populations

The scatter plot is one of the most underused exploratory tools. A mid-size dataset often takes only a second to plot, and a single scatter can reveal patterns — non-linear curves, clusters, outliers, censoring at a maximum value — that no summary statistic would surface.

5.9 Correlation

Correlation quantifies the strength and direction of the linear relationship between two continuous variables.

The **Pearson correlation coefficient** r :

$$r = (\sum (x - \bar{x})(y - \bar{y})) / ((n-1) \cdot s_x \cdot s_y)$$

r ranges from -1 to +1:

r value	Interpretation
+1	Perfect positive linear relationship
+0.7 to +1	Strong positive

r value	Interpretation
+0.4 to +0.7	Moderate positive
-0.4 to +0.4	Weak or none
-0.7 to -0.4	Moderate negative
-1 to -0.7	Strong negative
-1	Perfect negative linear relationship

```
# Single correlation
df["units_sold"].corr(df["revenue"])

# Full correlation matrix
df[["revenue", "units_sold", "discount_pct"]].corr()
```

The thresholds in the table above are rules of thumb, not laws. In a clean physics experiment, $r = 0.7$ might be considered weak. In a social science survey, $r = 0.4$ might be considered impressive. Context matters.

Visualising the Correlation Matrix

```
corr = df[["revenue", "units_sold", "discount_pct"]].corr()

sns.heatmap(corr, annot=True, cmap="coolwarm", center=0, fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

The heatmap uses colour to communicate direction (warm = positive, cool = negative) and annotation to show the exact value. For a DataFrame with a dozen numeric columns, the correlation heatmap is often the single most informative chart you can produce — it is a structured map of which variables move together and which do not.

Correlation Is Not Causation

A high correlation between two variables does not mean one causes the other. Both could be driven by a third variable, or the relationship could be coincidental. Ice cream sales correlate strongly with drowning rates, not because ice cream causes drowning but because both rise in summer. Correlation is a starting point for investigation, not a conclusion.

A second caution: Pearson correlation only captures *linear* relationships. Two variables can be perfectly determined by one another and still have r near zero, if the relationship is curved (a U-shape, for example). Always look at the scatter plot, not just the correlation number.

5.10 Subplots

When you want multiple charts together, use `plt.subplots()`.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# First chart
axes[0].hist(df["revenue"], bins=20)
axes[0].set_title("Revenue")

# Second chart
axes[1].hist(df["units_sold"], bins=15)
axes[1].set_title("Units Sold")

plt.tight_layout()
plt.show()
```

`tight_layout()` prevents overlapping labels. For a 2x2 grid, index axes as `axes[row, col]`.

A grid of small charts (sometimes called "small multiples") is often more informative than a single complex chart. Showing the same metric across regions, products, or time periods with a uniform scale makes patterns leap out that would be invisible in a single overlaid plot.

5.11 Chapter Summary

Concept	Key Point
<code>df.info()</code> / <code>df.describe()</code>	First steps when loading new data
<code>isnull().sum()</code>	Count missing values per column
<code>fillna(median)</code>	Preferred for skewed numeric columns
Do not fill with zero	Zero means something — misrepresents unknowns
<code>df.loc[cond, col] = value</code>	Fix invalid values in place
<code>np.where()</code>	Vectorised if-else for calculated columns
<code>.str</code> accessor	String methods applied to whole columns
Histogram	Shows distribution shape and skew
KDE	Smoothed distribution curve
Boxplot	Five-number summary — good for comparing groups
Scatter plot	Visualises relationship between two continuous variables
Pearson r	Linear correlation, -1 to +1
Heatmap	Colour-coded correlation matrix
<code>plt.subplots()</code>	Multiple charts in one figure
Correlation $\neq$ causation	Always investigate further before claiming a cause

Review Questions

1. A column `price` has 12 missing values out of 200 rows. The distribution of prices is right-skewed due to a few luxury items. Would you fill with mean or median? Justify your answer.
2. What is the difference between a missing value and an invalid value? Give one example of each from a business sales dataset.
3. A histogram shows the revenue distribution has a long right tail. Based on this, would you expect the mean to be higher or lower than the median? Explain.
4. Two boxplots show revenue by region. Region A has a tall box; Region B has a short box. What does this tell you about the sales teams in each region?
5. A scatter plot of advertising spend (x) and revenue (y) shows a strong positive correlation of $r = 0.82$. A colleague concludes that increasing advertising spend causes revenue to increase. What caution would you offer about this interpretation?

Chapter 6 — Grouped Analysis, Pivot Tables, and the EDA Workflow

The previous chapters built up the components of a complete analysis. This final chapter assembles them. The new techniques — grouping, pivoting, the coefficient of variation — are tools for asking questions about *segments* of a dataset, which is where most business questions actually live. The EDA workflow at the end of the chapter is the recipe for using everything you have learned to make sense of a dataset you have never seen before.

6.1 From Individual Rows to Group Summaries

The analyses in previous weeks operated on the dataset as a whole — overall mean, overall distribution, overall correlation. Most business questions are not about the whole; they are about segments: *Which region performs best? Does product mix vary by quarter? Which rep is most consistent?*

Answering these questions requires splitting the data into groups and summarising each group separately. Pandas makes this straightforward with `groupby`. The same idea exists in SQL as `GROUP BY` and in Excel as the pivot table — Pandas is, in many ways, "SQL with Python syntax" for the analyst's everyday work.

Most insight in business analytics comes from comparing things — quarter to quarter, region to region, segment to segment. A single overall mean is rarely the answer to a real question. The grouped versions of the statistics from earlier chapters are where the actionable information lives.

6.2 Grouped Analysis with `groupby`

`groupby` follows a **split-apply-combine** pattern:

1. **Split** the DataFrame into groups based on a categorical column

2. **Apply** an aggregation function to each group
3. **Combine** the results into a new DataFrame

```
import pandas as pd

data = {
    "rep":      ["Chen", "Patel", "Okafor", "Rivera", "Thompson",
                 "Chen", "Patel", "Okafor", "Rivera", "Thompson"],
    "region":   ["Northeast", "West", "South", "Midwest", "Northeast",
                 "Northeast", "West", "South", "Midwest", "Northeast"],
    "quarter":  ["Q1", "Q1", "Q1", "Q1", "Q1", "Q2", "Q2", "Q2", "Q2", "Q2"],
    "revenue":  [42100, 38900, 45300, 31800, 39200,
                 44500, 41200, 43800, 35600, 42100],
}
df = pd.DataFrame(data)

print(df.groupby("region")["revenue"].mean())
```

Output:

```
region
Midwest      33700.0
Northeast    41975.0
South        44550.0
West         40050.0
Name: revenue, dtype: float64
```

This reads as: *group the rows by region, then calculate the mean revenue for each group*. The result is a Series indexed by the group keys.

Applying Multiple Statistics — `.agg()`

`.agg()` applies several functions at once and lets you name the output columns.

```
print(df.groupby("region")["revenue"].agg(
    count = "count",
    total = "sum",
    mean   = "mean",
    median = "median",
    std    = "std"
))
```

Output:

	count	total	mean	median	std
region					
Midwest	2	67400	33700.0	33700.0	2687.005769
Northeast	4	167900	41975.0	42100.0	2236.069562
South	2	89100	44550.0	44550.0	1060.660172
West	2	80100	40050.0	40050.0	1626.345596

This is the workhorse pattern for any "summary by category" question. One call produces a publication-ready table.

Grouping by Multiple Columns

Pass a list to `groupby` to create groups defined by combinations of categories.

```
print(df.groupby(["rep", "quarter"])["revenue"].mean())
```

Output:

```
rep    quarter    revenue
Chen   Q1         42100.0
       Q2         44500.0
Okafor Q1         45300.0
       Q2         43800.0
Patel  Q1         38900.0
       Q2         41200.0
Rivera Q1         31800.0
       Q2         35600.0
Thompson Q1        39200.0
         Q2        42100.0
Name: revenue, dtype: float64
```

This returns the mean revenue for every rep-quarter combination — a natural way to track performance over time by individual. With more rows per combination, the same call would average over multiple records and reveal trends.

6.3 The Coefficient of Variation

When comparing spread across groups with different means, absolute standard deviation is misleading. A group with a higher mean will naturally tend to have a higher standard deviation in the same units, even if it is proportionally no more variable.

The **coefficient of variation (CV)** expresses standard deviation as a percentage of the mean, making it scale-independent.

$$CV = (s)/(\bar{x}) \times 100$$

```
region_stats = df.groupby("region")["revenue"].agg(mean="mean", std="std")
region_stats["cv_pct"] = (region_stats["std"] / region_stats["mean"] * 100).round(1)
print(region_stats)
```

Output:

```
region    mean    std  cv_pct
Midwest   33700.0  2687.005769    8.0
Northeast 41975.0  2236.069562    5.3
South     44550.0  1060.660172    2.4
West      40050.0  1626.345596    4.1
```

A region with a mean of 50,000 and std of 8,000 has $CV = 16\%$. A region with mean 30,000 and std of 6,000 has $CV = 20\%$. Despite the lower absolute std, the second region is more variable relative to its own baseline.

CV is useful for comparing consistency across reps, regions, or time periods — especially when their revenue scales differ. It is also dimensionless, which means you can compare a CV calculated in dollars with a CV calculated in pounds or euros — convenient for multinational reporting.

A practical caveat: CV is undefined when the mean is zero, and it is unstable when the mean is close to zero. It works well for ratio-scale data with positive values (revenue, counts, durations) and is a poor choice for data that includes zero or negative values.

6.4 Pivot Tables

A **pivot table** cross-tabulates two categorical variables and fills each cell with an aggregated value.

```
print(df.pivot_table(
    values = "revenue",
    index = "region",      # row categories
    columns = "quarter",   # column categories
    aggfunc = "mean"       # aggregation function
))
```

Output:

quarter	Q1	Q2
region		
Midwest	31800.0	35600.0
Northeast	40650.0	43300.0
South	45300.0	43800.0
West	38900.0	41200.0

The result is a matrix where each cell contains the mean revenue for a specific region-quarter combination. This format makes it easy to scan for patterns — which region-quarter had the highest mean, where there might be seasonal effects, which combinations are weak.

A pivot table with two grouping columns is conceptually identical to `groupby([col1, col2]).mean()` — but the pivot layout, with one variable as rows and the other as columns, is much easier to read for a two-way comparison. Use whichever fits the question.

Common Aggregation Functions

aggfunc	What It Computes
"mean"	Average value per cell
"sum"	Total value per cell
"count"	Number of observations per cell
"median"	Median value per cell

You can also pass a list of functions: `aggfunc=["mean", "std"]` produces both in a single table. Or pass a custom function (any callable) to compute something not in the standard list.

Adding Row and Column Totals

Pass `margins=True` to include row and column totals automatically.

```
print(df.pivot_table(values="revenue", index="region",
                     columns="quarter", aggfunc="sum", margins=True))
```

Output:

quarter	Q1	Q2	All
region			
Midwest	31800.0	35600.0	67400
Northeast	81300.0	86600.0	167900
South	45300.0	43800.0	89100
West	38900.0	41200.0	80100
All	197300.0	207200.0	404500

The bottom-right cell (`All / All`) is the grand total. Margins are convenient for sanity-checking sums and for handing the table directly to a stakeholder.

Visualising a Pivot Table

A pivot table is a matrix — a heatmap is its natural visual representation.

```
import seaborn as sns
import matplotlib.pyplot as plt

pivot = df.pivot_table(values="revenue", index="region",
                      columns="quarter", aggfunc="mean").round(0)

sns.heatmap(pivot, annot=True, fmt=".0f", cmap="YlGnBu", linewidths=0.5)
plt.title("Mean Revenue by Region and Quarter")
plt.show()
```

The colour gradient lets the eye quickly find the highest and lowest cells, while the annotations give the exact values. For a region-by-quarter sales matrix with a dozen rows and four columns, the heatmap conveys at a glance what would take a stakeholder a full minute to extract from the equivalent table of numbers.

6.5 The EDA Workflow

Exploratory data analysis (EDA) is a systematic process for understanding a new dataset before drawing conclusions or building models. The term was coined by John Tukey in 1977 — the same Tukey behind boxplots and IQR fences — and it remains the standard early-stage methodology for serious data work.

EDA combines everything from this course into a structured sequence. The point is not to follow it mechanically; the point is to make sure no obvious step gets skipped on the way to a finding.

The Five-Step EDA Workflow

Step	Goal	Key Questions
1. Structure	Understand what you have	How many rows and columns? What are the data types?
2. Data quality	Find and fix problems	Missing values? Invalid entries? Duplicates?
3. Univariate analysis	Understand each variable	What is the distribution shape? Mean vs median? Any outliers?
4. Bivariate analysis	Find relationships	How do pairs of variables relate? What correlates with the outcome?
5. Grouped analysis	Find segment patterns	Do relationships or distributions differ across categories?

Step 1 — Structure

```
print(df.shape)
print(df.dtypes)
df.head()
df.describe()
```

The first thirty seconds with a new dataset answer: how big, how wide, what types, what values. Surprises here — an unexpected column type, a row count off by an order of magnitude — often reveal upstream problems before you waste time on analysis.

Step 2 — Data Quality

```
df.isnull().sum()
(df["discount_pct"] < 0).sum()
df.duplicated().sum()
```

Always fix quality issues before any analysis. Findings built on dirty data are unreliable, and worse, they are confidently unreliable — they look correct, so they tend to slip into reports without scrutiny.

Step 3 — Univariate Analysis

For each variable:

- **Numeric:** histogram, mean, median, std, IQR, outlier check
- **Categorical:** frequency table, bar chart, most/least common

The histogram and the mean-vs-median comparison answer the same question from two angles — is the distribution symmetric or skewed? Are they consistent with each other?

For categorical variables, a frequency table answers most of the question, but a bar chart can reveal patterns the table obscures (such as a long tail of rare categories that should be lumped into "Other").

Step 4 — Bivariate Analysis

- Scatter plots for pairs of numeric variables
- Correlation matrix and heatmap
- Boxplots of a numeric variable split by a categorical variable

Bivariate analysis is where most actionable insights live. The biggest wins usually come from finding a variable that separates the outcome of interest cleanly — discount level that correlates with churn, region that correlates with margin, time of day that correlates with conversion. The correlation heatmap and a few well-chosen scatter plots are usually enough.

Step 5 — Grouped Analysis

- `groupby` with `.agg()` for mean, std, CV by category
- Pivot tables for two-way breakdowns
- Visualise with boxplots and heatmaps

Grouped analysis confirms or refines the patterns spotted in earlier steps and quantifies them in a way that translates into recommendations.

Mini Case Study — Capstone EDA

A retail chain has nine months of transaction data and asks for a "mid-year health check." Working through the EDA workflow:

1. **Structure.** The dataset has 1.3 million rows and 12 columns. Three columns are dates parsed as strings — they need conversion before any time-based analysis works correctly.
2. **Quality.** A small fraction of rows have negative `units_sold`, all from a single store on a single day — likely a register reconciliation event. About 4% of rows have missing `discount_pct`; the missing rows are concentrated on B2B accounts where discounts are negotiated and the field is meaningfully blank.
3. **Univariate.** Revenue is right-skewed as expected. The median transaction is 42 and the mean is 61 — typical retail. The discount distribution is bimodal: a peak near 0% and another near 20%, suggesting two distinct customer types are blended in the data.
4. **Bivariate.** Revenue and units sold correlate strongly ($r \approx 0.78$) but not perfectly, because some products are dramatically more expensive than others. Discount level correlates weakly negatively with margin ($r \approx -0.32$), as expected.
5. **Grouped.** A region-by-month pivot reveals one region has flat year-on-year growth while every other region grew 8–14%. That single finding becomes the headline of the report.

The whole workflow — top to bottom — takes a competent analyst less than a day on a dataset of this size. The output is a one-page summary with a handful of charts and a clear recommendation. None of the techniques used were more advanced than what is in this book.

6.6 From Analysis to Narrative

Numbers are not findings. A finding is a number plus an interpretation:

Just a number	A finding
Northeast mean = \$46,200	The Northeast region leads all regions in mean revenue, outperforming the next closest (West at \$43,800) by approximately 5%.
Correlation = 0.03	Discount level has no meaningful linear relationship with revenue in this dataset.
CV (South) = 22%	The South region is the most variable — nearly twice the CV of the Midwest (12%) — suggesting inconsistent performance that may warrant investigation.

Business analysts are expected to translate numbers into sentences that a non-technical stakeholder can act on. The statistics provide evidence; the narrative provides meaning.

A good finding has three parts: the *what* (the number or the pattern), the *so what* (why it matters in business terms), and the *now what* (what action it suggests). A correlation of 0.03 is a *what*; "discount level does not predict revenue" is a *so what*; "we can stop tracking discount level as a leading indicator and look elsewhere for revenue drivers" is a *now what*. Stakeholders are most grateful for analysts who do all three.

A second skill: knowing when not to over-interpret. A small absolute difference between groups, even if statistically detectable, may be operationally irrelevant. Conversely, a finding that fails a statistical significance test may still be worth flagging if it is consistent with prior knowledge and the cost of investigation is low. Both calls require judgment, and both improve with practice.

6.7 Choosing the Right Summary

A quick reference for the decision points covered across all six weeks:

Question	Tool
What is the centre of the data?	Mean (symmetric data), Median (skewed or outliers present)
How spread out is the data?	Std deviation, IQR
How does spread compare across groups with different scales?	CV
Is a value unusually far from the rest?	Z-score, IQR fences
How often does each category appear?	Frequency table, Counter
How does a variable distribute across its range?	Histogram
How do groups compare on a numeric variable?	Boxplot
Is there a relationship between two numeric variables?	Scatter plot, Pearson r

Question	Tool
How does a numeric variable break down across two categories?	Pivot table, heatmap

6.8 Chapter Summary

Concept	Key Point
<code>groupby</code>	Split by category, apply a function, combine results
<code>.agg()</code>	Multiple statistics in one call with named output columns
CV	$s / \bar{x} \times 100$ — relative spread, comparable across groups
Pivot table	Two-way cross-tabulation — rows, columns, aggregated values
<code>margins=True</code>	Adds row and column totals to a pivot table
Heatmap of pivot	Natural visual for a cross-tabulated matrix
EDA workflow	Structure → quality → univariate → bivariate → grouped
Analysis narrative	Interpret numbers in plain language — what, so what, now what

Review Questions

1. What does the `groupby` "split-apply-combine" pattern mean? Describe each step in plain language.
2. Two sales regions have the following statistics. Which is more consistent, and why? - Region A: mean = 52,000, std = 9,100 - Region B: mean = 38,000, std = 7,200
3. You create a pivot table showing total revenue by product (rows) and quarter (columns). One cell shows a very high value compared to the others. What are two possible explanations, and how would you investigate further?
4. Write the Pandas code to create a summary table showing mean, median, and standard deviation of `units_sold` grouped by `rep`, with the results sorted by mean in descending order.
5. You present your EDA findings to the sales director. She points to your highest-performing region and says: "Let's just focus all our resources there." What statistical information from your analysis would you use to give a more nuanced recommendation?

This is the final chapter. You have completed Introduction to Python for Business Statistics.

What Comes Next

This course built a foundation. Where it leads depends on your goals:

Direction	Next Steps
More statistics	Hypothesis testing, regression, probability distributions
More Python	Object-oriented programming, APIs, automation
Data engineering	SQL, databases, data pipelines
Machine learning	scikit-learn, feature engineering, model evaluation
Visualisation	Advanced Seaborn, Plotly for interactive charts, dashboards

The EDA workflow from this course is the starting point for all of them. Hypothesis testing extends Chapter 4 with formal questions about whether observed differences are real. Regression extends Chapter 5's correlation into a model that makes predictions. Machine learning builds on the cleaning and feature engineering instincts that come from doing EDA well. The fundamentals are the same; the toolkit just gets larger.

License

Creative Commons Attribution 4.0 International (CC BY 4.0)

© 2026 Patrick Dolinger

This work is licensed under the Creative Commons Attribution 4.0 International License.

You are free to:

- **Share** — copy and redistribute the material in any medium or format
- **Adapt** — remix, transform, and build upon the material for any purpose

Under the following terms:

- **Attribution** — You must give appropriate credit to the author, provide a link to the license, and indicate if changes were made.